A Corrected Metatheory for Mergeable Replicated Data Types

Anonymous Authors

Abstract

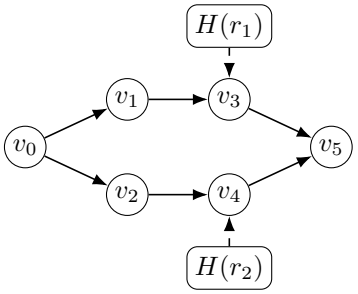
Mergeable replicated data types reconcile two branch states relative to a common ancestor. The published bottom-up linearization strategy for this model is unsound: a global arbitration relation can lose an edge when an event outside the version being linearized absorbs it. We give a corrected Lean 4 metatheory based on set-relative replay, a store-wide reachable invariant, and a contextual ternary Join lemma. We then connect replay to an independent sequential specification through explicit issuance and interaction certificates. To make immutable version histories practical, we give an asynchronous distributed collector that preserves compressed reachability and lowest-common-ancestor answers without retaining the root or ancestor paths. A separate interface covers garbage collection inside a retained datatype state, and a trace-erasure theorem composes the two collectors. The paper states the operational semantics, verification conditions, and refinement theorems in full; the cited Lean declarations are machine-checked evidence, not a substitute for those statements.

Contents

1 Problem, result, and evidence boundary

An MRDT stores immutable versions in a Git-like directed acyclic graph. An update extends one replica head. A merge of branch states a and b reads the state l at their common ancestor and computes $\mathsf{merge}(l, a, b)$. The client expects the state at every registered version to equal a legal sequential fold of exactly that version's events.

Figure 1 is the running execution shape. The graph is not decoration: apply creates a one-parent version, merge creates a two-parent version, and a later merge may use an old version as its base. This is why the correctness invariant ranges over the store rather than only the current heads.



$$S_5 = \mathsf{merge}(S_0, S_3, S_4)$$
$$E_5 = E_3 \cup E_4$$

Figure 1: A version-DAG execution. Solid arrows point from parents to children. The merge at v_5 reads the registered LCA v_0 and both branch heads.

The result has three capstones. First, the corrected Join theorem establishes per-version convergence for ordinary and virtual-LCA execution. Second, a sequential certificate gives every retained version a legal abstract history with the same queries. Third, two independent collection protocols refine the uncollected semantics, both separately and in composition. Table 1 states who supplies each component.

Layer	Framework supplies	Datatype supplies
Raw execution	version DAG, apply, merge, query	state, update, merge, query
Convergence	canonical adequacy, virtual LCA	contextual Join certificate
Client protocol	certified transition schema	origin issuance relation
Client meaning	witness theorem schema	interaction and sequential proofs
Commit GC	distributed protocol and refinement	none
State GC	generic trace-erasure theorem	optional physical protocol

This boundary matters. A theorem about a semantic MRDT does not by itself verify an editor widget, a wire decoder, or crash recovery. The JavaScript runtime is tested against the model-facing fixtures, but it is not extracted from Lean. Its evidence manifest names the exact Lean package associated with each released datatype and labels representation experiments separately.

Evidence convention. We call a proposition accepted by Lean *machine-checked*, an executable runtime correspondence test *validated*, and a benchmark observation *measured*. The trusted model

includes Lean's kernel, Mathlib definitions, and the paper-to-source transcription. It does not include the JavaScript runtime: that implementation is not extracted from Lean. The accompanying claim ledger records a source declaration for every load-bearing statement.

2 Raw MRDT semantics

2.1 The signature stays plain

Definition 2.1 (MRDT signature)**.** An MRDT signature D contains a state type Σ, an initial state σ_0, application-operation type O, query type Q, result type R, and total functions

$$\mathsf{do} : \Sigma \to (\mathbb{N} \times \mathbb{N} \times O) \to \Sigma, \quad \mathsf{query} : \Sigma \to Q \to R, \quad \mathsf{merge} : \Sigma \to \Sigma \to \Sigma \to \Sigma.$$

An event $e = (t, r, o)$ contains a Lamport timestamp, replica identifier, and application operation. In $\mathsf{merge}(l, a, b)$, l is the common-ancestor state and a, b are the branch states. Thus an implementer supplies one merge function, not separate binary and ancestor-aware implementations. `MRDTSig`

The raw signature contains no invariant, applicability predicate, issuance guard, conflict relation, arbitration policy, or convergence proof. Its functions are executable and total. Sections 4 and 5 add the proof and client layers without changing this transition system. Lean's older replay library accepts a binary CRDT signature, so the mechanization derives its internal merge as $\lambda a\ b.\, \mathsf{merge}(\sigma_0, a, b)$. This is a projection, not another field or proof obligation.

2.2 Configurations and transitions

A paper-level configuration is the projection

$$C = \langle V, H, P, \mathsf{vis} \rangle, \qquad V(v) = (S_v, E_v),$$

where V is a partial version store, H maps replicas to heads, P maps a version to its parents, and vis is event visibility. Every finite execution allocates only finitely many entries of V. Replica state and replica events are derived as $S_{H(r)}$ and $E_{H(r)}$. The Lean `Configuration` also caches these two projections as `N` and `L`; `head_coherent` requires the caches to equal the derivation above. We omit the caches from the rules because they contain no independent semantic information.

The remaining well-formedness fields are not omitted:

1. every visibility endpoint occurs in an active event set, visibility is causally closed at each active replica, and same-replica events are visible in one direction;
2. distinct stored events have distinct timestamps, and $e_1\ \mathsf{vis}\ e_2$ implies $t(e_1) < t(e_2)$;
3. parent versions have smaller version numbers, version 0 stores $(\sigma_0, \varnothing)$, and every active head names its cached state and events;
4. if v_T is an LCA of stored v_1, v_2, then

$$\mathcal{E}(v_T) = \mathcal{E}(v_1) \cap \mathcal{E}(v_2).$$

The last condition is semantic bookkeeping, not a property assumed of an arbitrary Git DAG. It is preserved by the transition rules and later derived from the store invariant used in the adequacy proof.

An event is a triple $e = (t, r, o)$ of timestamp, issuer, and application operation. Applying a fresh event extends visibility from every event at the issuer head to e:

$$\mathsf{vis}' = \mathsf{vis} \cup \big(\mathcal{E}(H(r)) \times \{e\}\big).$$

Merge does not add an event and therefore leaves visibility unchanged.

Figure 2 gives all four constructors of `Step`. The notation $C[v \mapsto (S, E); r \mapsto v; P(v) \mapsto \pi]$ updates the version store, replica head, and parent list while preserving all other entries. The raw Apply rule deliberately does not check whether the operation was issuable at that replica. Write $\mathsf{freshTime}(C, t)$ when no event in an active replica set or any stored version has timestamp t; both clauses appear separately in Lean because an old version need not be an active head.

$$\frac{H(r) = \bot}{C \xrightarrow{\mathsf{create}(r)} C[r \mapsto 0]}\ \textsc{Create} \qquad \frac{H(r) = v \quad V(v) = (S, E) \quad z = \mathsf{query}(S, q)}{C \xrightarrow{\mathsf{query}(r,q,z)} C}\ \textsc{Query}$$

$$\frac{\begin{array}{c} H(r) = v \quad V(v) = (S, E) \quad \mathsf{freshTime}(C, t) \\ v' \notin \mathsf{dom}(V) \quad v < v' \quad e = (t, r, o) \end{array}}{C \xrightarrow{\mathsf{apply}(t,r,o)} C[v' \mapsto (\mathsf{do}(S, e), E \cup \{e\}); r \mapsto v'; P(v') \mapsto [v]]}\ \textsc{Apply}$$

$$\frac{\begin{array}{c} H(r_1) = v_1 \quad H(r_2) = v_2 \quad \mathsf{LCA}_P(v_1, v_2) = v_T \\ V(v_i) = (S_i, E_i)\ (i \in \{1, 2, T\}) \quad v_m \notin \mathsf{dom}(V) \quad v_1, v_2 < v_m \end{array}}{C \xrightarrow{\mathsf{merge}(r_1,r_2)} C[v_m \mapsto (\mathsf{merge}(S_T, S_1, S_2), E_1 \cup E_2); r_1 \mapsto v_m; P(v_m) \mapsto [v_1, v_2]]}\ \textsc{Merge}$$

Figure 2: Paper-level raw operational semantics. The Lean rules also update the coherent replica caches. Create points a fresh replica at the shared initial version; Query is read-only.

The initial configuration has one active replica, numbered 0, at version 0. Create activates another replica at the same initial version. Apply adds visibility edges from the issuer's entire current event set to the new event. Merge writes its result at r_1 and leaves r_2 unchanged. These choices remove three common ambiguities: replica creation is not a state copy, merge is not broadcast, and merge introduces no event or visibility edge.

3 Why the earlier induction fails

The earlier proof equipped the update semantics with a proof-local replay policy and derived an arbitration order lo. This policy is not part of the raw datatype and should not be confused with the client interaction policy in Section 5. The failed induction computed lo from the entire configuration while linearizing a smaller version event set. A third event outside that set can then absorb an arbitration edge that the smaller history still needs.

Write $e_1 \parallel_C e_2$ when neither event sees the other, and write $e_1\ \mathsf{rc}\ e_2$ when the historical proof's conflict resolver chooses e_1 before e_2. The repaired relation is indexed by the event set being linearized:

$$\begin{aligned} e_1\ \mathsf{lo}_E\ e_2 \iff & \big(e_1\ \mathsf{vis}\ e_2 \wedge \neg(e_1 \rightleftarrows e_2)\big) \\ & \vee \big(e_1 \parallel_C e_2 \wedge e_1\ \mathsf{rc}\ e_2 \wedge \neg\exists e_3 \in E.\ e_2\ \mathsf{vis}\ e_3 \wedge \neg(e_2 \rightleftarrows e_3)\big). \end{aligned}$$

Here $e_1 \rightleftarrows e_2$ means that their updates commute. The final existential is the important repair: only an absorber inside E may erase an arbitration edge while E is being linearized.

Proposition 3.1 (global-order convergence is false)**.** *There is a two-operation add-wins signature and a legal configuration with a backward-closed event set E such that two enumerations of E both respect the configuration-global arbitration relation but fold to different states.* `convergence_over_backward_closed_subsets_false`

The falsifier contains concurrent noncommuting e_1, e_2 and an event e_3 with e_2 vis e_3. The global relation sees e_3 and removes the replay edge $e_1 \to e_2$. A version containing only $E = \{e_1, e_2\}$ cannot use e_3 to justify that removal, so both list orders appear admissible although their folds differ. The checked countermodel is in `Metatheory/Join/Convergence_CounterModel.lean`. The repaired `loOn C E` retains the edge because its absorber quantifier ranges only over E.

Associativity, commutativity, and idempotence of merge do not repair the proof. The second checked countermodel satisfies the core update laws and a bounded join-semilattice merge but violates both the peel laws and the Join lemma. `coreVCs_lattice_insufficient`

These countermodels establish two boundaries: arbitration must be local to the set being linearized, and convergence needs a contextual bridge between merge and update history.

Figure 3 summarizes the repair. The failed path tried to peel operations from a merge using a global order. The checked path instead proves canonicality for each stored event set and uses one contextual Join obligation at every merge.

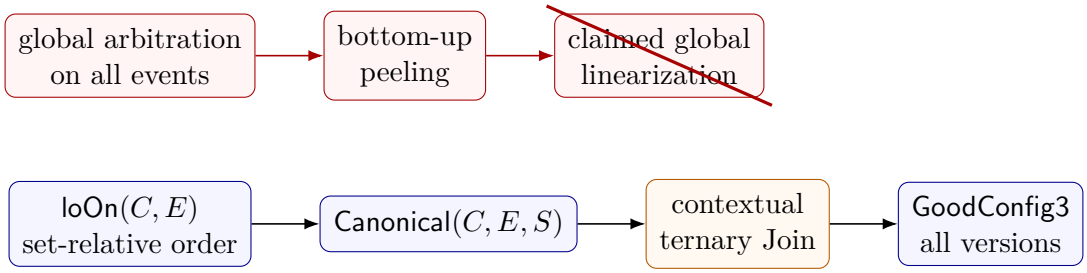


Figure 3: The failed bottom-up argument and the checked canonical-adequacy argument. Red nodes describe the refuted route; blue/orange nodes are current Lean interfaces and invariants.

4 Canonical adequacy

4.1 Canonical states

For a list $\pi = [e_1, \ldots, e_n]$, let $\mathsf{perm}(\pi, E)$ mean that π enumerates exactly E without duplicates. Let $\mathsf{respects}(\pi, R)$ mean that no later list element is required by R to precede an earlier element. Canonicality is then the following state predicate:

$$\mathsf{Canonical}(C, E, s) \iff \exists \pi.\ \mathsf{perm}(\pi, E) \wedge \mathsf{respects}(\pi, \mathsf{lo}_E) \wedge \mathsf{fold}(\sigma_0, \pi) = s. \qquad \text{(Canonical)}$$

This is `IsCanonicalState`. The update-layer lemmas prove that any two such enumerations have the same fold under the supplied update conditions.

The reachable invariant `GoodConfig3` states that every registered version contains a canonical state of its event set. It also retains visibility transitivity and irreflexivity, event support, and causal closure. The invariant ranges over all versions, not only replica heads, because later merges may read an older version as their LCA.

The public correctness target uses the original configuration-wide order lo_C. The set-relative relation is stronger on E, so a canonical witness also satisfies the public target:

$$\mathsf{RALin}(C) \iff \forall v, s, E.\ V(v) = (s, E) \Rightarrow \exists \pi.\ \mathsf{perm}(\pi, E) \wedge \mathsf{respects}(\pi, \mathsf{lo}_C) \wedge \mathsf{fold}(\sigma_0, \pi) = s. \tag{RA-Lin}$$

Thus the strengthened invariant is internal to the corrected proof; the observable theorem remains per-version RA-linearizability.

4.2 The ternary Join obligation

The hypotheses of Join are easy to understate, so we name them explicitly. Let $\mathsf{WF}(C)$ mean that vis is transitive and irreflexive. For an event set E, define

$$\mathsf{Supported}_C(E) \equiv \forall e \in E.\ e \in C.\mathsf{events},$$
$$\mathsf{WeakClosed}_C(E) \equiv \forall a, b.\ a\ \mathsf{vis}\ b \wedge \neg(a \rightleftarrows b) \wedge b \in E \Rightarrow a \in E.$$

Weak closure retains visible predecessors only when their updates do not commute. Full causal closure drops the noncommutation premise.

Definition 4.1 (contextual ternary Join)**.** $\mathsf{JoinAt}(D, C)$ holds when, for all E_1, E_2, s_0, s_1, s_2,

$$\mathsf{WF}(C),\quad \mathsf{Supported}_C(E_1),\quad \mathsf{Supported}_C(E_2),\quad \mathsf{WeakClosed}_C(E_1),\quad \mathsf{WeakClosed}_C(E_2),$$
$$\mathsf{Canonical}(C, E_1 \cap E_2, s_0),\quad \mathsf{Canonical}(C, E_1, s_1),\quad \mathsf{Canonical}(C, E_2, s_2)$$

imply

$$\mathsf{Canonical}\,(C, E_1 \cup E_2, \mathsf{merge}(s_0, s_1, s_2))\,. \tag{Join}$$

`JoinLemma3At`

`JoinLemma3` is $\forall C.\mathsf{JoinAt}(D, C)$. A datatype whose proof depends on honest issuance instead proves $\mathsf{MintHonest}(C) \Rightarrow \mathsf{JoinAt}(D, C)$. The alternative `JoinLemma3F` changes both weak-closure premises to full causal closure; the framework's reachable invariant supplies that stronger fact.

Theorem 4.2 (ordinary adequacy)**.** *If a datatype satisfies `JoinLemma3`, every configuration reachable from `initConfig` by `Step` is RA-linearizable at every registered version.* `ra_linearizable3_of_join`

The proof carries two invariants. `StoreInv` says that parents are allocated, event sets grow along parent edges, and every event has an allocated origin version that reaches every version containing it. The origin property proves the non-obvious half of $E_T = E_1 \cap E_2$: a shared event's origin reaches both branches, hence reaches their LCA. `GoodConfig3` says that every stored version is canonical, every stored event set is supported and causally closed, and visibility is transitive and irreflexive. Apply appends its fresh event to a canonical fold; merge invokes Join with the LCA equality; query and create preserve both invariants.

The framework retains the eight-condition route as one constructor for Join: `CoreVCs3CD`, `FeasibleDeltaVCs3`, and `CDVC3` imply `JoinLemma3`. Datatypes may instead prove Join directly; the public `VerifiedMRDT` package depends on the convergence result, not on one particular VC decomposition.

4.3 A checkable eight-obligation route to Join

Join is the semantic convergence obligation. The framework offers a sufficient decomposition for delta-shaped ternary merges; an implementer may instead prove Join directly. We state the decomposition fully because its contextual premises are exactly what an unconditional algebraic account loses.

First fix a proof-local replay policy $\mathsf{rc}(e_1, e_2) \in \{\mathsf{first}, \mathsf{second}, \mathsf{either}\}$. Write $e \# e'$ for distinct timestamps and $\mathsf{rep}(e) \neq \mathsf{rep}(e')$ for different issuers. The three `UpdateVCs` obligations are:

$$e_1 \# e_2 \wedge \mathsf{rep}(e_1) \neq \mathsf{rep}(e_2) \Rightarrow \big(\neg(e_1 \rightleftarrows e_2) \iff \mathsf{rc}(e_1, e_2) = \mathsf{first} \vee \mathsf{rc}(e_2, e_1) = \mathsf{first}\big), \tag{U1}$$

$$e_1 \# e_2 \wedge e_2 \# e_3 \Rightarrow \neg\big(\mathsf{rc}(e_1, e_2) = \mathsf{first} \wedge \mathsf{rc}(e_2, e_3) = \mathsf{first}\big), \tag{U2}$$

$$\mathsf{rc}(e, e') = \mathsf{first} \wedge \neg(e' \rightleftarrows e'') \Rightarrow \mathsf{do}(\mathsf{fold}(\mathsf{do}(\mathsf{do}(s, e'), e), \pi), e'') = \mathsf{do}(\mathsf{fold}(\mathsf{do}(\mathsf{do}(s, e), e'), \pi), e''). \tag{U3}$$

Equation (U3) also assumes pairwise-distinct e, e', e''. These obligations belong to this reusable proof constructor, not to `MRDTSig` or the public client interface. `CoreVCs3CD` packages (U1)–(U3) and branch symmetry

$$\mathsf{merge}(l, a, b) = \mathsf{merge}(l, b, a). \tag{M1}$$

To state the remaining four obligations, define

$$\downarrow_C e = \{x \mid x = e \vee \mathsf{TransGen}(\lambda a\, b.\ a \ \mathsf{vis}\ b \wedge \neg(a \rightleftarrows b))\ x\ e\}$$

and let $\mathsf{Adm}(C, E_1, E_2, e)$ abbreviate:

1. $\mathsf{WF}(C)$, support and weak closure of E_1, E_2;
2. $e \in E_1 \cup E_2$; and
3. union maximality, $\forall x \in E_1 \cup E_2.\ x \neq e \Rightarrow \neg(e\ \mathsf{lo}_{E_1 \cup E_2}\ x)$.

The feasible unit law is

$$\mathsf{Supported}_C(E) \wedge \mathsf{Canonical}(C, E, s) \Rightarrow \mathsf{merge}(\sigma_0, \sigma_0, s) = s. \tag{M2}$$

For the local redistribution law, assume $\mathsf{Adm}(C, E_1, E_2, e)$, $e \in E_1$, $e \notin E_2$, and

$$\mathsf{Canonical}(C, E_1 \cap E_2, s_0), \quad \mathsf{Canonical}(C, \downarrow_C e \setminus \{e\}, B),$$
$$\mathsf{Canonical}(C, E_1 \setminus \{e\}, t_1), \quad \mathsf{Canonical}(C, E_2, s_2).$$

Then, writing $B{+}e = \mathsf{do}(B, e)$,

$$\mathsf{merge}\big(s_0, \mathsf{merge}(B, t_1, B{+}e), s_2\big) = \mathsf{merge}\big(B, \mathsf{merge}(s_0, t_1, s_2), B{+}e\big). \tag{M3}$$

For shared redistribution, assume $\mathsf{Adm}(C, E_1, E_2, e)$, $e \in E_1 \cap E_2$, and canonical states

$$t_0 : (E_1 \cap E_2) \setminus \{e\}, \quad B : \downarrow_C e \setminus \{e\}, \quad t_1 : E_1 \setminus \{e\}, \quad t_2 : E_2 \setminus \{e\}.$$

The required equation is

$$\mathsf{merge}\big(\mathsf{merge}(B, t_0, B{+}e), \mathsf{merge}(B, t_1, B{+}e), \mathsf{merge}(B, t_2, B{+}e)\big) = \mathsf{merge}\big(B, \mathsf{merge}(t_0, t_1, t_2), B{+}e\big). \tag{M4}$$

Equations (M2)–(M4) form `FeasibleDeltaVCs3`. "Feasible" is load-bearing: every state named above is a canonical state of the exact event set at that position in the proof.

Finally, `CDVC3` considers any supported, weakly closed U and lo_U-maximal $e \in U$. If

$$\mathsf{Canonical}(C, U \setminus \{e\}, A), \qquad \mathsf{Canonical}(C, {\downarrow_C}\, e \setminus \{e\}, B),$$

then

$$\mathsf{merge}(B, A, B{+}e) = A{+}e. \tag{M5}$$

Theorem 4.3 (VC adequacy)**.** *Obligations (U1)–(U3) and (M1)–(M5), equivalently* `CoreVCs3CD`*,* `FeasibleDeltaVCs3`*, and* `CDVC3`*, imply* `JoinLemma3`*, which in turn implies per-version RA-linearizability of every reachable raw configuration.* `join_lemma3_of_cd_feasible`*,* `ra_linearizable3_of_join`

Why these equations suffice. Choose a $\mathsf{lo}_{E_1 \cup E_2}$-maximal event. If it is local to one side, (M5) peels the inner causal delta, (M3) moves it through the outer merge, and the induction hypothesis joins the smaller event sets. If it is shared, (M5) peels it from the LCA and both branches, and (M4) moves the shared delta out once. (M2) closes the empty case; (M1) exchanges branches. The update laws justify swapping adjacent replay events and guarantee that the maximal-event decomposition is well founded. This is the checked proof structure of `join_lemma3_of_cd_feasible`; the displayed obligations do not claim necessity.

5 Issuance, sequential meaning, and safety

RGA insertion illustrates why a plain signature still needs a client protocol. An insertion carries an anchor and a fresh identifier. Raw delivery can apply that record anywhere, but honest minting requires the issuer to have seen the anchor and to choose an identifier above its causal past. This is a historical fact about the minting state, not a shape predicate on an arbitrary later delivery state.

Definition 5.1 (issuance)**.** An `Issuance D` contains only

$$\mathsf{CanIssue} \;\; : \mathsf{Op} \to \Sigma \to \mathsf{Prop}.$$

`IssuedStep` checks `CanIssue` only for an apply transition and checks it at the issuer's current materialized head. Non-apply transitions retain their raw semantics. Define the visible past of event e by $\mathsf{Past}_C(e) = \{x \in C.\mathsf{events} \mid x \;\mathsf{vis}\; e\}$. The persistent honesty assertion is

$$\mathsf{MintHonest}_I(C) \equiv \forall e \in C.\mathsf{events}.\; \exists \pi.\; \mathsf{perm}(\pi, \mathsf{Past}_C(e)) \wedge \mathsf{respects}(\pi, \mathsf{vis}) \wedge I.\mathsf{CanIssue}(e, \mathsf{fold}(\sigma_0, \pi)). \tag{Mint}$$

A final configuration does not store all former issuer heads. Consequently, `MintCertifiedReach` records that (Mint) held before and after every issued step rather than attempting to reconstruct a deleted origin state from the final heads. `MintCertifiedReachV` is the virtual-LCA counterpart. A datatype derives any additional history invariant from this witness; the public interface does not accept an arbitrary invariant field.

$$\frac{C \xrightarrow{\mathsf{apply}(t,r,o)} C' \quad H(r) = v \quad V(v) = (S, E) \quad I.\mathsf{CanIssue}((t, r, o), S)}{C \xRightarrow[I]{\mathsf{apply}(t,r,o)} C'} \textsc{Certified-Apply}$$

$$\frac{C \xrightarrow{\ell} C' \quad \ell \neq \mathsf{apply}(-,-,-)}{C \xRightarrow[I]{\ell} C'} \textsc{Certified-Other}$$

Figure 4: Issuance-certified execution. Certification restricts minting at the issuer head; it does not alter raw delivery or merge semantics. A `MintCertifiedReach` trace additionally retains the historical mint witness needed after the issuer moves to a later head.

Definition 5.2 (sequential specification)**.** A `SequentialSpec D` supplies an abstract state, initial state, deterministic step, query function, and

$$\mathsf{Legal} : \mathsf{List}(\mathsf{Op}) \to \mathsf{Prop}.$$

The legality predicate sees only the abstract event history. It cannot inspect the implementation state. A total datatype sets it to true.

Definition 5.3 (semantic interaction)**.** An `InteractionSpec D` classifies each ordered event pair as *independent* or as a *conflict*. A conflict carries one concurrent verdict: first-before-second, second-before-first, or unconstrained. Reversing the event pair must flip the verdict. This policy refers to abstract events, not arbitrary representation states.

For a version event set E, `interactionLoOn` orders every visible conflict by visibility. It orders a concurrent conflict only when the policy selects first-before-second and the second event has no later conflicting absorber inside E. This absorber clause belongs to the public witness order; it does not expose the proof-local replay resolver.

Two finite checks explain why this separation matters. LWW orders concurrent writes by increasing Lamport timestamp. Three writes therefore form a two-edge chain, which refutes the historical `no_rc_chain` requirement (`InteractionSPOT.LWW.old_no_chain_refuted`). For add-wins observed-remove semantics, concurrent remove and add conflict and the policy orders remove before add. If the remove observed the add, visibility instead orders add before remove. The same policy therefore explains both add-wins concurrency and ordinary observed removal.

Definition 5.4 (client-facing RA-linearizability)**.** Given interaction policy A, sequential specification S, and representation relation $\mathsf{Rel} \subseteq \Sigma \times S.\mathsf{State}$, $\mathsf{SpecRALin}(D, A, S, \mathsf{Rel}, C)$ holds when every $V(v) = (s, E)$ admits a list π such that

$$\begin{aligned} &\mathsf{perm}(\pi, E), \qquad \mathsf{respects}(\pi, \mathsf{interactionLoOn}(A, C, E)), \\ &S.\mathsf{Legal}(\pi), \qquad \mathsf{Rel}(s, S.\mathsf{run}(\pi)), \\ &\forall q. \quad D.\mathsf{query}(s, q) = S.\mathsf{query}(S.\mathsf{run}(\pi), q). \end{aligned} \tag{Spec-RA}$$

`IsSpecRALinearizable`

The first line constrains the explanation's events and ordering. The second rules out a witness that the sequential ADT itself considers illegal. The third rules out a representation relation that relates states but fails to preserve observations. Internal replay linearizability establishes none of these extra clauses automatically.

Definition 5.5 (verified MRDT)**.** An implementer supplies:

- an issuance relation I;
- an interaction policy A;
- a convergence certificate proving internal replay linearizability for every I-certified virtual-LCA execution;
- an independent sequential specification S and representation relation Rel; and
- one sequential-correctness certificate proving (Spec-RA) from any ordinary or canonical-virtual certified execution and its replay theorem.

`VerifiedMRDT`

Definition 5.6 (production package)**.** A `PackagedMRDT` is a dependent pair (D, V) where D is one raw `MRDTSig` and V has type `VerifiedMRDT D`. The typed `Production.registry` contains only these pairs. A raw signature, replay-only certificate, or theorem about a different signature cannot enter the registry.

The package exposes ordinary and virtual-LCA versions of (Spec-RA) as `VerifiedMRDT.converges` and `VerifiedMRDT.convergesV`. The replay-only `ReplayVerifiedMRDT` remains an internal migration and negative-analysis device; it is not the public correctness result. A datatype may separately supply a `SafetyCertificate`. Safety remains separate because convergence does not imply a client invariant. For example, the bounded counter proves non-negative observable values through a global history invariant; arbitrary ternary state tuples do not preserve that invariant.

The executable `CRDTSig` contains no resolver. The historical absorber-based convergence construction receives a proof-local `ReplayPolicy`. The generic certified Join route fixes its low-priority, unconstrained default; `IsRALinearizableWith` remains an internal hook for studying another replay proof. Consequently, neither `rc` nor `no_rc_chain` is an implementer-facing MRDT field.

6 Canonical virtual LCAs

Criss-cross histories can have several maximal common ancestors and no unique registered LCA. Choosing one maximal ancestor can lose events represented by another. The framework instead constructs a synthetic state by recursively folding the maximal-common-ancestor frontier.

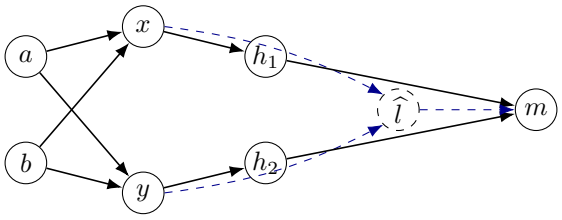


Figure 5: The registered DAG has two maximal common ancestors, x and y. The resolver constructs $S_{\hat{l}} = \mathsf{foldMCA}(\{x, y\})$ for event set $\mathcal{E}(x) \cup \mathcal{E}(y)$; only merge result m is allocated. Solid arrows are stored parent edges; blue dashed arrows describe the ghost computation.

`canonicalVirtualLCA` implements this resolver. The support decreases under the version order, which justifies the recursion. The key theorem `virtualLCAState_canonical` proves that

the synthetic state is canonical for the union of the maximal ancestors' event sets. The store theorem `mca_events_cover` shows that this union covers every common ancestor.

`StepV` conservatively embeds every ordinary `Step` and adds one virtual merge constructor. For $V(v_i) = (s_i, E_i)$ and resolver $\widehat{S}(C, v_1, v_2)$, that rule is

$$\frac{\begin{array}{c} H(r_i) = v_i \ (i = 1,2) \quad V(v_i) = (s_i, E_i) \\ v_m \notin \mathsf{dom}(V) \quad v_1, v_2 < v_m \end{array}}{\begin{array}{l} C \xrightarrow{\mathsf{merge}(r_1, r_2)} C', \\ V'(v_m) = (\mathsf{merge}(\widehat{S}(C, v_1, v_2), s_1, s_2), E_1 \cup E_2), \\ H'(r_1) = v_m, \qquad P'(v_m) = [v_1, v_2] \end{array}} \textsc{Virtual-Merge}$$

It allocates only the result version. The maximal-common-ancestor frontier and its recursively merged state are ghost semantic values, not virtual commits that a runtime must store or later garbage-collect.

Theorem 6.1 (virtual-LCA adequacy)**.** *The same `JoinLemma3` that proves ordinary adequacy proves RA-linearizability for configurations reachable under `StepV` with the canonical resolver.* `ra_linearizable3V_of_join`

The proof first establishes canonicality of $\widehat{S}$ for the union of the MCA event sets. It then applies the same Join premise used by ordinary merge. Therefore virtual LCAs introduce no new datatype merge law; they strengthen only the framework's construction of the base state.

7 Distributed commit-history garbage collection

Commit-history GC belongs to the runtime framework. It is not supplied once per datatype, and the public framework has no global stop-the-world collector. The specification baseline is the same asynchronous fetch protocol with the collection step erased. This gives a direct refinement statement and avoids a fictional globally synchronized intermediate machine.

7.1 Minimal local state

For each replica, the logical collector stores only

$$L = \{\mathsf{head} : \mathit{Version}, \quad \mathsf{commits} : \mathcal{P}(\mathit{Version})\}, \qquad \mathsf{head} \in \mathsf{commits}.$$

The parent relation, fixed roster, and immutable optional author of each version are protocol parameters. Apply-created versions may record their origin replica; roots and merge versions are unauthored. Authorship is used to establish progress evidence, not authenticity against an adversary. An envelope contains only a commit set. Fetch is set union and does not change the receiving head; head synchronization is a separate transition. `GC.Local`, `GC.Envelope`

A distributed world is only a replica-indexed family of these local records:

$$W : \mathit{Replica} \to \mathit{Local}.$$

In particular, the logical protocol does not store a second frontier map, an authorship index, or a copy of the global configuration at each replica. Efficient implementations may cache indexes, but the proof does not require them as semantic state.

Evidence is derived rather than stored. Replica r has frontier evidence at L when L retains a commit authored by r that reaches $L.\mathsf{head}$. Formally,

$$\mathsf{evidence}(L, r) = \{v \in L.\mathsf{commits} \mid \mathsf{author}(v) = r \wedge v \preceq L.\mathsf{head}\}.$$

`GC.EvidenceComplete` requires such evidence for every other roster member. The negative control `GC.EvidenceSPOT.missing_author` confirms that an unauthored merge head cannot stand in for a missing author's evidence.

7.2 Collection certificate

A `GC.Certificate` supplies a keep set and a compressed reachability relation $\preceq_K$. It contains the completeness proof, retains the local head and one derived witness for every remote roster member, proves support in the local holding, and preserves exact reachability and LCA answers between retained versions. The canonical constructor `GC.Certificate.ofMCAClosed` takes K closed under maximal common ancestors and instantiates $\preceq_K$ with reachability in the original graph restricted to retained endpoints.

For local record L and keep set K, the certificate obligations are:

$$\mathsf{EvidenceComplete}(L), \qquad L.\mathsf{head} \in K, \qquad K \subseteq L.\mathsf{commits},$$
$$\forall r \in \mathsf{roster}.\ r = \mathsf{self} \ \vee\ \exists v \in K.\ v \in \mathsf{evidence}(L, r),$$
$$a, b \in K \Rightarrow \big(a \preceq_K b \iff a \preceq b\big),$$
$$v_1, v_2, v_T \in K \Rightarrow \big(\mathsf{LCA}_K(v_1, v_2, v_T) \iff \mathsf{LCA}(v_1, v_2, v_T)\big).$$

The compressed graph connects two retained versions when the original graph reached between them. It need not retain intermediate paths or the old root. Figure 6 shows the distinction between retaining a path and retaining its reachability fact.

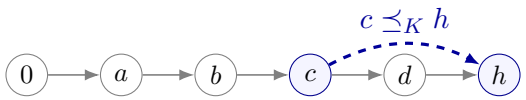


Figure 6: Compressed history retains endpoint answers, not the root or the intermediate path. Branching histories additionally retain the maximal common ancestors required by future LCA queries.

`GC.compressedReaches_iff` proves reachability equivalence, and `GC.root_absent_when_dropped` proves that a root outside the keep set is absent from the retained carrier. Keeping only the heads is insufficient: two retained heads may have a maximal common ancestor that collection would otherwise erase, changing the next merge base. Keeping the entire ancestor spine is sufficient but unnecessary; MCA closure is the smaller semantic condition.

7.3 Direct asynchronous refinement

The collecting protocol has fetch, head synchronization, and local GC. Its no-GC counterpart has the same network steps and no collection transition. The simulation relates one full and one compact local store by equal heads and commit-set inclusion:

$$\mathsf{StoreSim}(F, C) \ \equiv\ F.\mathsf{head} = C.\mathsf{head} \ \wedge\ C.\mathsf{commits} \subseteq F.\mathsf{commits} \ \wedge\ C.\mathsf{head} \in C.\mathsf{commits}.$$

$$\frac{M = \mathsf{commits}(W(s))}{W \longrightarrow W[d \mapsto \langle \mathsf{head}(W(d)), \mathsf{commits}(W(d)) \cup M \rangle]}\ \textsc{Fetch}$$

$$\frac{v \in \mathsf{commits}(W(r))}{W \longrightarrow W[r \mapsto \langle v, \mathsf{commits}(W(r)) \rangle]}\ \textsc{Head-Sync} \qquad \frac{\mathsf{Certificate}(W(r), K)}{W \longrightarrow W[r \mapsto \langle \mathsf{head}(W(r)), K \rangle]}\ \textsc{Collect}$$

Figure 7: Asynchronous commit-history GC. Fetch is the ordinary remote-fetch path and unions immutable commits. Head synchronization is explicit. Collect is local and requires complete frontier evidence plus reachability/LCA preservation for the keep set.

Let NoGCStep contain only Fetch and Head-Sync. The one-step simulation is

$$\mathsf{WorldSim}(F, C) \wedge C \longrightarrow_{\mathrm{GC}} C' \Longrightarrow \exists F'.\ F \longrightarrow_{\mathrm{NoGC}}^{0 \text{ or } 1} F' \wedge \mathsf{WorldSim}(F', C').$$

The collection case chooses $F' = F$; network cases take their matching no-GC step. Induction over the finite collecting trace gives the execution theorem.

Theorem 7.1 (distributed collection refines no-GC)**.** *Every finite collecting execution is matched by a finite no-GC execution. Fetch and head synchronization match their counterparts; collection stutters. The final worlds have equal replica heads, so head-state reads are unchanged.* `GC.execution_refines_noGC`, `GC.read_preserved`

This store theorem deliberately says only that an already-taken collecting trace has a no-GC match. To connect stores to the datatype semantics, define a proof-level runtime $R = \langle C, W \rangle$. Well-formedness says every locally held version is allocated in C, and every semantic head agrees with and is present in its local holding. A visible step additionally proves that its required versions are available: ordinary merge needs both heads and its LCA; a virtual merge needs the resolver's declared read set. Visible steps contain the actual `Step` or `StepV` derivation and install the new head in the local holding. Fetch and collection leave C unchanged.

Theorem 7.2 (runtime history-GC refinement)**.** *If* $R_0 \xRightarrow[\mathsf{runtime}]{\lambda_1 \ldots \lambda_n} R_n$*, then erasing silent fetch and collection labels yields*

$$R_0.C \xRightarrow[\mathsf{Step}]{\mathsf{filterMap}(\lambda_1 \ldots \lambda_n)} R_n.C.$$

The analogous result holds for `StepV` *and an explicit virtual-resolver read set.* `GC.runtime_refines_core`, `GC.runtime_refines_coreV`

The core configuration is ghost simulation state, not a second physical copy required of an implementation. The theorem also makes the collector's trust boundary concrete: malformed network data and Byzantine authorship are out of scope; commits and authorship are immutable protocol facts.

8 Optional datatype-state garbage collection

Commit GC removes versions from a replica's history. It cannot decide whether an RGA tombstone, a Peritext mark boundary, or a TreeMove event can be removed from the state stored at a retained version. That decision is datatype specific.

Definition 8.1 (state-GC protocol)**.** A `StateGCProtocol D V` supplies

$$\begin{aligned} \mathsf{Physical} &: \mathsf{Type}, \\ \mathsf{semantic} &: \mathsf{Physical} \to \mathsf{Configuration}(D), \\ \mathsf{Valid} &: \mathsf{Physical} \to \mathsf{Prop}, \\ \mathsf{PhysicalStep} &: \mathsf{Physical} \to \mathsf{Option}(\mathsf{Label}) \to \mathsf{Physical} \to \mathsf{Prop}. \end{aligned}$$

It proves:

1. validity is invariant;
2. a silent physical transition leaves the semantic projection unchanged;
3. a visible transition refines a `StepV` transition.

Theorem 8.2 (state collection refines semantic execution)**.** *Erasing silent labels from any valid finite physical trace yields a finite* ***`StepV`*** *trace between the projected configurations.* *`StateGCProtocol.refines`*

The combined physical state contains the datatype protocol state and the generic distributed commit stores. A combined transition is either a datatype physical step or a silent fetch/commit-GC step. Writing $P = \langle p, W \rangle$, the two cases are

$$\frac{p \xrightarrow{\lambda}_S p' \quad W \xrightarrow{\lambda}_{\mathsf{store}} W'}{\langle p, W \rangle \xrightarrow{\lambda} \langle p', W' \rangle}\ \textsc{Datatype} \qquad \frac{W \xrightarrow{\tau}_{\mathsf{fetch/GC}} W'}{\langle p, W \rangle \xrightarrow{\tau} \langle p, W' \rangle}\ \textsc{History}.$$

For $\lambda = \tau$, a datatype step leaves the commit store unchanged. For a visible label, its store evolution installs the version created by the semantic step. Both history transitions are silent at the datatype semantics.

Theorem 8.3 (the two collectors compose)**.** *For every valid finite combined trace, erasing state-GC, fetch, and commit-GC labels yields an execution of the uncollected virtual-LCA semantics.* *`GC.CombinedSteps.refinesV`*

The ordinary corollary `GC.CombinedSteps.refinesRaw` applies when every visible datatype transition proves a raw `Step`. Sided Peritext and TreeMove provide concrete state-GC protocols. Other datatypes may omit the interface entirely.

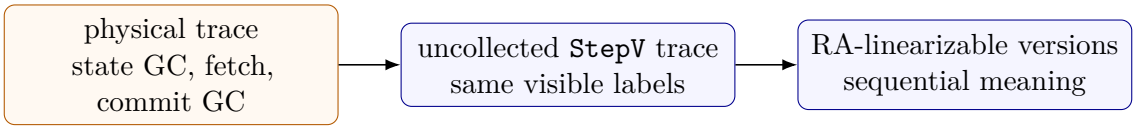


Figure 8: The two collectors remove different objects and compose by trace erasure. Commit GC removes versions; datatype GC removes metadata inside a retained state. The first arrow is `CombinedSteps.refinesV`; the second uses the `VerifiedMRDT` capstones.

9 Machine-checked instances

Table 1 lists the packages imported by `Metatheory/RefactorLedger.lean`. A checkmark means that the current Lean tree contains the named public package; it does not claim that the JavaScript runtime is extracted from that package.

Datatype	Issuance	Convergence	Public spec	State GC
Bounded counter	✓	✓	✓	–
RGA	✓	✓	✓	–
EmbedRGA	✓	✓	✓	–
SidedEmbedRGA	✓	✓	✓	–
Observed-remove set	✓	✓	✓	–
Peritext/RichCore	✓	✓	✓	concrete protocol
TreeMove	✓	✓	✓	concrete protocol
AegisSheet	✓	✓	✓[1]	certificate

Table 1: Current nontrivial production packages. [1]The AegisSheet theorem uses causal-origin legality because the old whole-prefix legality rejects valid concurrent origins; its reference state is causally aware, not only the visible sheet. Trivial-generation grow-only and counter products are also in the ledger.

The breadth matters because the obligations are not tailored to text. The bounded counter uses nontrivial origin issuance; RGA and Peritext use honest fresh identifiers and anchors; the observed-remove set uses a directional public interaction policy; TreeMove and AegisSheet carry hidden identities needed by later conflict handling.

AegisSheet supplies a useful negative abstraction result: `no_view_only_step` exhibits equal visible sheets whose responses to the same next operation differ. Its independent sequential machine must therefore retain causal identities rather than only visible cells. This is also why origin issuance and merged-history legality are separate: concurrent operations may each be legal at their own causal origin even when neither is legal after the other in a whole-prefix check.

External implementation correspondence. We validated the public AegisSheet Scala implementation against its published intent matrices and found four focused undo/range regressions at the audited revision. These are implementation-correspondence observations, not Lean counterexamples to the merge kernel. The reproducible audit and its revision boundary are in `docs/aegissheet-scala-audit.md`.

The separate negative ledger checks intentionally refuted or incomplete claims, including MVR/single-register and queue/FIFO counterexamples. FugueMax's coordinate lemmas justify the ordering policy used by the verified SidedEmbedRGA package but do not create a standalone production datatype. The release gate rejects `sorry` and `sorryAx`, requires every production entry to inhabit `PackagedMRDT`, and checks the distributed and composed GC capstones.

10 Reproduction and residual boundary

From the repository root, run the production proof and runtime gate with `./scripts/check-mrdt-refactor.sh`. Build both papers and the declaration ledger with `./scripts/check-working-papers.sh`.

The proved framework covers semantic MRDT executions, canonical virtual LCAs, distributed commit-history collection, optional state-GC protocols, and their composition. Same-replica crash recovery is not yet part of the verified or tested continuation protocol. The JavaScript implementation uses indexes and cached metadata for efficiency; the paper-facing logical GC state remains the minimal $\{\mathsf{head}, \mathsf{commits}\}$ record.